

Compiler Design

Laboratory Manual(V1.3)

Subject Code: CS44114

Semester: IV

Year: 2015-16 (Spring Semester) (Rev. 2021-22, 2024-25)

MARCH 24, 2025

Faculty:

Dr. Bhaskar Mondal

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY PATNA
Patna, Bihar, India- 800005

Contents

1	Design of Lexical Analyzer	2
1.1	Introduction	2
1.2	Day 1: Using C	4
1.3	Day 2: Introduction to LEX	4
2	Scanning input Symbols	7
2.1	Special Variables and Procedures	7
2.2	Exercise on Scanning input Symbols and counting them	7
3	Recognizing simple Lexemes	7
4	Recognizing simple Lexemes and count them	8
5	Some useful tasks on patterns and string recognitions	8
6	Reading input from a source file	8
6.1	Scanner for a Pascal-like language	8
7	Parser	9
7.1	Exercise using C	9
7.2	Introduction to YACC	9
7.3	Compiling the Program on YACC	10
7.4	Exercise using Lex and YACC	11
7.5	Calculator using YACC	11
7.6	Day: 6: Exercise using YACC	14
7.7	Day 7: Exercise Using C	14
8	Syntax Directed Definition	14
8.1	Day 8: Exercise Using C	14
9	Generation of Intermediate Code	14
9.1	Day 9: Exercise Using C	14
10	Advanced Problems	15
11	Books to Follow	15

Instructions

- Maintain Index/ content properly.
- Brief descriptions including **algorithm used and flowchart** of the work you did for each exercise.
- Copies of the main **C, Lex and Yacc** you used for the exercises, along with the numerical results.
- You must calculate and mention **computational Complexity** of each experiment.
- You must provide **Test Cases/sample Input and Output** at the end of exercise, Print the plot files (.jpg) corresponding with the different exercises(if any).
- Explanations of anything unusual or interesting, or points of confusion that you were unable to resolve outside lab.
- If you believe I have an error in a lab, please inform me of it. Explain why you think it is an error and, if you like, suggest a correction.

1 Design of Lexical Analyzer

1.1 Introduction

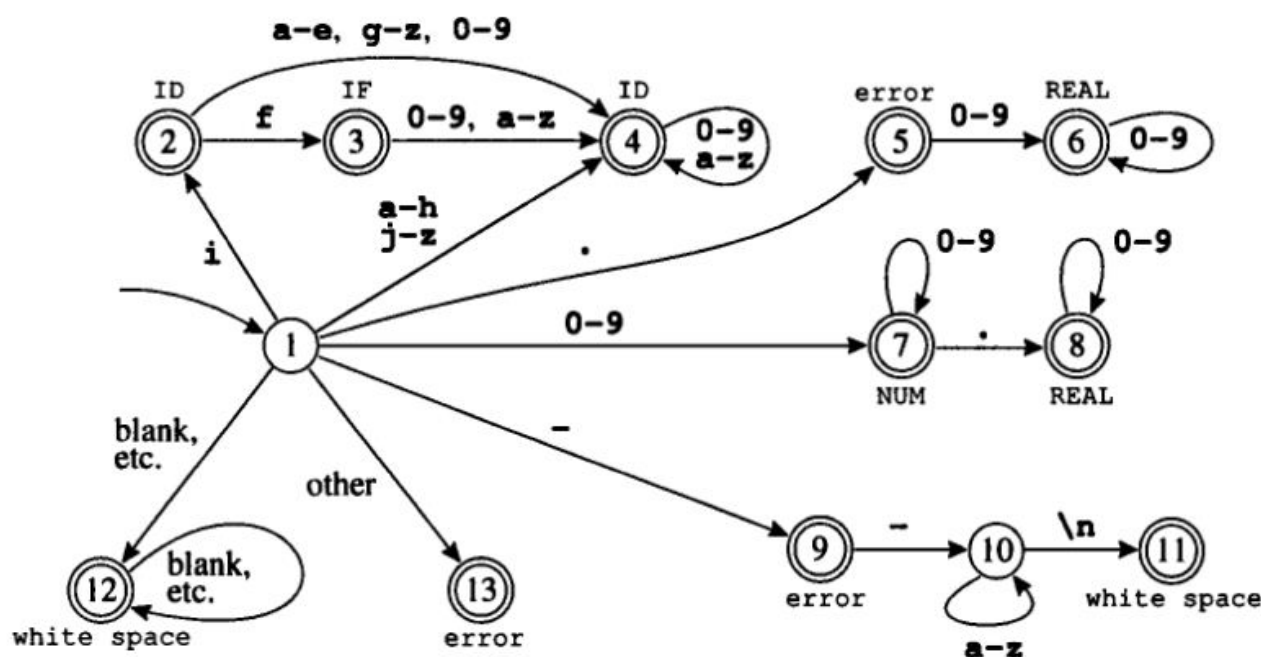


Figure 1

The patterns in the input are written using an extended set of regular expressions. These are:

'x'	match the character 'x'
'.'	any character (byte) except newline
'[xyz]'	a "character class"; in this case, the pattern matches either an 'x', a 'y', or a 'z'
'[abj-oZ]'	a "character class" with a range in it; matches an 'a', a 'b', any letter from 'j' through 'o', or a 'Z'
'[^A - Z]'	a "negated character class", i.e., any character but those in the class. In this case, any character EXCEPT an uppercase letter.
'[^A - Z\ ⁿ]'	any character EXCEPT an uppercase letter or a newline
'r*'	zero or more r's, where r is any regular expression
'r+'	one or more r's
'r?'	zero or one r's (that is, "an optional r")
'r2,5'	anywhere from two to five r's
'r2,''	two or more r's
'r4'	exactly 4 r's
'name'	the expansion of the "name" definition (see above)
""[xyz]foo""	the literal string: '[xyz]"foo'
'\x'	if x is an 'a', 'b', 'f', 'n', 'r', 't', or 'v', then the ANSI-C interpretation of x. Otherwise, a literal 'x' (used to escape operators such as '*')
'\0'	a NUL character (ASCII code 0)
'\123'	the character with octal value 123
'\x2a'	the character with hexadecimal value 2a
'(r)'	match an r; parentheses are used to override precedence (see below)
'rs'	the regular expression r followed by the regular expression s; called "concatenation"
'r—s'	either an r or an s
'r/s'	an r but only if it is followed by an s. The text matched by s is included when determining whether this rule is the longest match, but is then returned to the input before the action is executed. So the action only sees the text matched by r. This type of pattern is called trailing context. (There are some combinations of 'r/s' that flex cannot match correctly; see notes in the Deficiencies / Bugs section below regarding "dangerous trailing context".)
'^r'	an r, but only at the beginning of a line (i.e., which just starting to scan, or right after a newline has been scanned).
'r\$'	an r, but only at the end of a line (i.e., just before a newline). Equivalent to "r/\ ⁿ ". Note that flex's notion of "newline" is exactly whatever the C compiler used to compile flex interprets '\ ⁿ ' as; in particular, on some DOS systems you must either filter out \ ⁿ 's in the input yourself, or explicitly use $r/\backslash r\backslash n$ for "r\$".
'< s > r'	an r, but only in start condition s (see below for discussion of start conditions) ;s1,s2,s3; r same, but in any of start conditions s1, s2, or s3
'< * > r'	an r in any start condition, even an exclusive one.
'<< EOF >>'	an end-of-file ;s1,s2; EOF; an end-of-file when in start condition s1 or s2

Note that inside of a character class, all regular expression operators lose their special meaning except escape ('\ⁿ') and the character class operators, '-', ']', and, at the beginning of the class, '^'. What is a token

lexeme	token
sum	IDENT
=	ASSIGN_OP
3	NUMBER
+	ADD_OP
2	NUMBER
;	SEMICOLON

1.2 Day 1: Using C

1. Write a C Program to Design Lexical Analyzer which will identify keywords, identifiers, sentinels, special characters, operators, number of lines in code.
2. Write down a program in C to identify a input as id/ keywords/ number, the program should able to take a line of instruction (int rate = 50;) and recognize all the small parts of the instruction.

1.3 Day 2: Introduction to LEX

% `sudo apt-get install flex` (in ubuntu)

<http://rpmfind.net/linux/rpm2html/search.php?query=flex>

Installation of *flex* on Windows Down load and Install Cygwin from <https://www.cygwin.com/>

Cygwin is:

- a large collection of GNU and Open Source tools which provide functionality similar to a Linux distribution on Windows.
- a DLL (cygwin1.dll) which provides substantial POSIX API functionality.

Use Command `Install gcc, make, gdb, flex ... package`

Installation of *flex* on Windows Installation Instructions

Step 1: Download FLEX

Step 2: Download DevC++

Step 3: Install FLEX in "C:\GnuWin32"

Step 4: Install DevC++ in "C:\Dev-Cpp"

Step 5: Open Environment Variables (Steps on how to get to environment variables is given below)

Step 6: Add this "C:\GnuWin32\bin;C:\Dev-Cpp\bin;" to PATH.

Step 7: Stop

How to set to Environment Variables in Windows

Step 1: Click Start

Step 2: Right Click "Computer"

Step 3: Click Properties

Step 4: When the window opens, Click on the Advanced Settings in the left pane

Step 5: Click on Environment Variables in the bottom

Step 6: Select path in the 2nd window and click edit and add the lines mentioned above

Compiling lex programs Lets assume that you have a lex program written under name first.l

Step 1: Open Command prompt

Step2: Type "flex first.l"

Step3: Type "cc lex.yy.x"

Step 4: type "a"

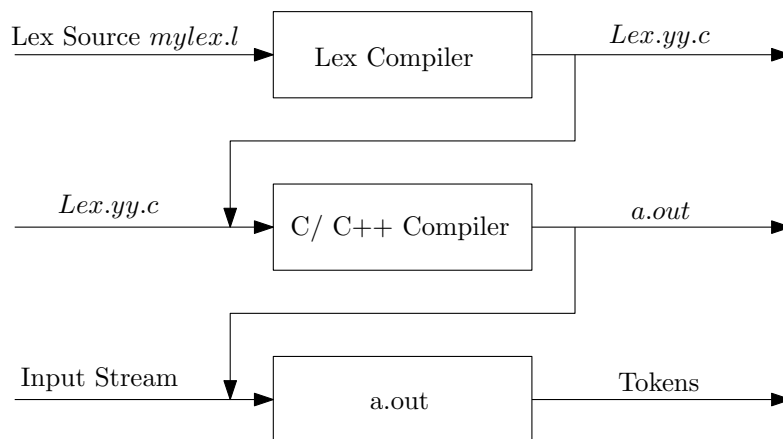


Figure 2

Compilation Here is the step by step method of compiling a LEX program.

```

% flex example.l (output file lex.yy.c)
% gcc lex.yy.c -lfl (-lfl : to link flex library )
    or
% gcc lex.yy.c -ll (-lfl : to link lex library )
  
```

Execution:

```

% cat input | ./a.out (in linux)
\ $ cat input | ./a.exe (in windows)
  
```

You can also use:

```
yyin=fopen("input","r");
```

Writing Lex Program: The Structure of a Lex Program

```

(Declarations)
%%
(Regular expression rules)
%%
(Subroutines definitions)
  
```

Declarations Lex copies the material between “%{“ and “%}” directly to the generated C file, so you may write any valid C codes here E.g.

```

%{
#define A 100
%}
WS [ \t ]+
letter [A-Za-z]
digit [0-9]
op_plus "+"
  
```

Regular expression rules Each rule is made up of two parts

- A pattern (regular expression)
- An action
- Lex had a set of simple disambiguating rules:
 - Lex patterns only match a given input character or string once
 - Lex executes the action for the longest possible match for the current input
- It can consists of any legal C code,
- Lex copies it to the C file after the end of the Lex generated code

```
[\\t ]+          /* ignore white space */ ;
{op_plus}       return OP_PLUS;
[a-zA-Z]+       { printf( "\\%s: is alpha\\n", yytext); }
. | \\n          { ECHO; /*_normal_default_anyway_*/ }

%%
```

Subroutines definitions The main function and other C functions if required.

```
main()
{
    yylex();
}
```

Create a Lex file, let's say "exp.l" and open it up in your favorite text editor (read: Notepad++)

```
%{
#define A 100
%}
WS [\\t]+
letter [A-Za-z]
digit [0-9]
op_plus "+"

%%
[0-9]+ {printf ("digit");};
%%

main() {
    yylex();
}
yywrap(void)
{
    return 0;
}
```

2 Scanning input Symbols

2.1 Special Variables and Procedures

yytext	where token text is stores
yytext	length of the token text
yylineno	the current line number
yywrap	A user function. It returns 1 when no more input to process, otherwise, return 0
int yywrap(void)	wrapup, return 1 if done, 0 if not done 0
int yylex(void)	call to invoke lexer, returns token
char *yytext	pointer to matched string
yytext	length of matched string
yyval	value associated with token
FILE *yyout	output file
FILE *yyin	input file
INITIAL	initial start condition
BEGIN	condition switch start condition
ECHO	write matched string

2.2 Exercise on Scanning input Symbols and counting them

1. Lex program to check whether input is digit or not.
2. identify if an input is a digit or alphabet
3. identify if a character apart from alphabets occurs in a string
4. Write a Lex program to count numbers of vowels and consonants
5. Code Section prints the number of capital letter present in the given input
6. Write a Lex program to count numbers of lines, and characters

```
int num_lines = 0, num_chars = 0;
```

```
%%
```

```
\n    ++num_lines; ++num_chars;
.      ++num_chars;
```

```
%%
```

```
main()
{
    yylex();
    printf( "#_of_lines_=_%d, #_of_chars_=_%d\n", num_lines, num_chars );
}
```

3 Recognizing simple Lexemes

1. Write a Lex program to recognize valid keywords.
2. Write a Lex program to determine whether input is an identifier or not
3. Write a Lex program to determine whether input is an valid operator (arithmetic and logical) or not
4. Write a Lex program to determine whether input is an integer or float.

4 Recognizing simple Lexemes and count them

1. Write a Lex program to count number of lexemes
2. Write a Lex program to count total number of digit, number, words, operators, command lines, spaces, etc.

5 Some useful tasks on patterns and string recognitions

1. LEX code to identify a string is a valid URL or not?
2. LEX code to identify a string is a valid Mobile Number or not?
3. LEX code to identify a string is a valid email or not?

6 Reading input from a source file

1. Lex program to print the total characters, white spaces, tabs and lines in the given input file.
2. Write a Lex program to count number of lexemes of the given word in a file
3. Write a Lex program to count total number of digit, number, words, operators, command lines, spaces, etc. of the given word in a file
4. Lex program to take input from file and remove multiple spaces, newline and tab and write output in a separate file
5. Program using LEX to count the numbers of comment lines in a given C program. Also eliminate them and copy the resulting program into separate file.

6.1 Scanner for a Pascal-like language

```
%{
/* need this for the call to atof() below */
#include <math.h>
}%

DIGIT      [0-9]
ID         [a-z][a-z0-9]*

%%

{DIGIT}+   {
    printf( "An_integer:_%s_(%d)\n", yytext,
            atoi( yytext ) );
}

{DIGIT}+"."{DIGIT}* {
    printf( "A_float:_%s_(%g)\n", yytext,
            atof( yytext ) );
}

if|then|begin|end|procedure|function {
    printf( "A_keyword:_%s\n", yytext );
}

{ID}       printf( "An_identifier:_%s\n", yytext );
```

```
"+"|"-"|"*"|"/"    printf( "An_operator:_%s\n", yytext );

"{"[^\\n]*}"        /* eat up one-line comments */

[ \\t\\n]+          /* eat up whitespace */

.                    printf( "Unrecognized_character:_%s\n", yytext );

%%

main( argc, argv )
int argc;
char **argv;
{
    ++argv, --argc; /* skip over program name */
    if ( argc > 0 )
        yyin = fopen( argv[0], "r" );
    else
        yyin = stdin;

    yylex();
}
```

7 Parser

7.1 Exercise using C

1. Write a C program to compute FIRST, FOLLOW of a given grammar.
2. Write a C program to compute FIRST, FOLLOW and look-ahead of a given grammar

7.2 Introduction to YACC

1. Install YACC. (Bison), bison-2.4.1-setup.exe <http://gnuwin32.sourceforge.net/packages/bison.htm>
2. Study of LALR parser generation by Yacc.
3. Generate SLR parser using Bison

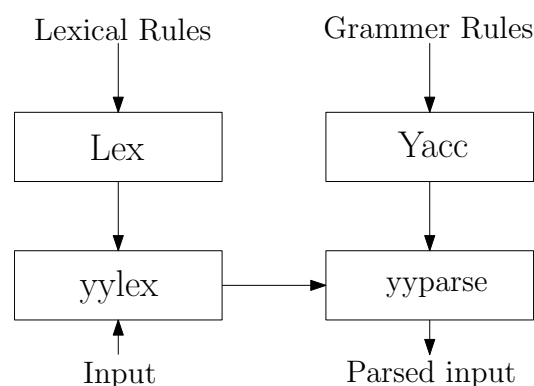


Figure 3

7.3 Compiling the Program on YACC

1. Process the yacc grammar file using the `-d` optional flag (which tells the yacc command to create a file that defines the tokens used in addition to the C language source code):

```
yacc -d hello.y OR bison -dy hello.y
```

2. Use the `li` command to verify that the following files were created:

- **y.tab.c** The C language source file that the yacc command created for the parser.
- **y.tab.h** A header file containing define statements for the tokens used by the parser.

3. Process the lex specification file:

```
lex hello.lex OR flex hello.l
```

4. Use the `li` command to verify that the following file was created:

lex.yy.c The C language source file that the lex command created for the lexical analyzer.

5. Compile and link the two C language source files **cc y.tab.c and lex.yy.c**

```
gcc lex.yy.c y.tab.c -o hello.exe
```

6. Use the `li` command to verify that the following files were created:

- **y.tab.o** The object file for the y.tab.c source file
- **lex.yy.o** The object file for the lex.yy.c source file
- **a.out** The executable program file (hello.exe in case of Window)
- To then run the program directly from the a.out file, enter:

```
\$ a.out
```

Lexical Analyzer Source Code

```
%{
    #include "y.tab.h"
    int yyerror(char *errmsg);
    int yyparse();
}%

%%

("hi"|"oi") "\n"      { return HI; }
("tchau"|"bye") "\n"  { return BYE; }
[-[+].,><]             { return yytext[0]; }
.                      { yyerror("Unknow_char"); }

%%

int main(void)
{
    yyparse();
    return 0;
}

int yywrap(void)
{
    return 0;
}

int yyerror(char *errmsg)
{
    fprintf(stderr, "%s\n", errmsg);
    exit(1);
}
```

YACC or Parser Source Code

```
%{  
  
#include <stdio.h>  
#include <stdlib.h>  
int yylex(void);  
int yyerror(const char *s);  
  
%}  
  
%token HI BYE  
  
%%  
  
program:  
    hi bye  
    ;  
  
hi:  
    HI      { printf("Hello_World\n"); }  
    ;  
  
bye:  
    BYE     { printf("Bye_World\n"); exit(0); }  
    ;
```

7.4 Exercise using Lex and YACC

1. Write a program using Lex and YACC which will print “Hello World” on input token “hi” and “Bye World” on input “bye”
2. Write a program using Lex and YACC to check whether given string is Palindrome or Not
3. Write a program using Lex and YACC which accept strings that starts and ends with 0 or 1
4. Write a program using Lex and YACC to check whether given number is Armstrong number or not
5. Write a program using Lex and YACC to recognize strings of $a^n b | n \geq 5$
6. Write a program using Lex and YACC to implement a simple calculator.

7.5 Calculator using YACC

When you start the program, the cursor moves to the line below the \$ (command prompt). Then enter numbers and operators in calculator fashion. When you press the Enter key, the program displays the result of the operation. After you assign a value to a variable:

```
m=4 ;enter;
```

the cursor moves to the next line. When you use the variable in subsequent calculations, it will have the assigned value:

```
m+5 ;enter;
```

9 Lexical Analyzer Source Code

Following are the contents of the calc.lex file. This file contains include statements for standard input and output, as well as for the y.tab.h file. The yacc program generates that file from the yacc grammar file information if you use the -d flag with the yacc command. The y.tab.h file contains definitions for the tokens that the parser program uses. In addition, calc.lex contains the rules to generate these tokens from the input stream.

```
%{  
  
#include <stdio.h>  
#include "y.tab.h"  
int c;  
extern int yylval;  
%}  
%%  
"_"      ;  
[a-z]    {  
    c = yytext[0];  
    yylval = c - 'a';  
    return(LETTER);  
}  
[0-9]    {  
    c = yytext[0];  
    yylval = c - '0';  
    return(DIGIT);  
}  
[^a-z0-9\b] {  
    c = yytext[0];  
    return(c);  
}
```

Parser Source Code The following example shows the contents of the calc.yacc file. This file has entries in all three sections of a yacc grammar file: declarations, rules, and programs.

```
%{  
#include <stdio.h>  
int regs[26];  
int base;  
%}  
%start list  
%token DIGIT LETTER  
%left '|'   
%left '&'   
%left '+' '-'   
%left '*' '/' '%'   
%left UMINUS /*supplies precedence for unary minus */  
%%  
/* beginning of rules section */  
list:                                     /*empty */  
    |  
    list stat '\n'  
    |  
    list error '\n'  
    {  
        yyerrok;  
    }  
    ;  
stat:  expr  
    {  
        printf("%d\n", $1);  
    }  
    |  
    LETTER '=' expr  
    {  
        regs[$1] = $3;  
    }  
    ;
```

```
expr:      '(' expr ')'  
        {  
            $$ = $2;  
        }  
        |  
        expr '*' expr  
        {  
            $$ = $1 * $3;  
        }  
        |  
        expr '/' expr  
        {  
            $$ = $1 / $3;  
        }  
        |  
        expr '%' expr  
        {  
            $$ = $1 % $3;  
        }  
        |  
        expr '+' expr  
        {  
            $$ = $1 + $3;  
        }  
        |  
        expr '-' expr  
        {  
            $$ = $1 - $3;  
        }  
        |  
        expr '&' expr  
        {  
            $$ = $1 & $3;  
        }  
        |  
        expr '|' expr  
        {  
            $$ = $1 | $3;  
        }  
        |  
        '-' expr %prec UMINUS  
        {  
            $$ = -$2;  
        }  
        |  
        LETTER  
        {  
            $$ = regs[$1];  
        }  
        |  
        number  
        ;  
number:  DIGIT  
        {  
            $$ = $1;  
            base = ($1==0) ? 8 : 10;  
        }  
        |  
        number DIGIT
```

```
        {  
            $$ = base * $1 + $2;  
        }  
    ;  
%%  
main()  
{  
    return(yyvsparse());  
}  
yyerror(s)  
char *s;  
{  
    fprintf(stderr, "%s\n", s);  
}  
yywrap()  
{  
    return(1);  
}
```

7.6 Day: 6: Exercise using YACC

1. Convert The BNF rules into Yacc form and write code to generate abstract syntax tree.
2. YACC program to recognize a valid arithmetic expression that uses operators +, -, * and /.
3. YACC program to recognize a valid variable, which starts with a letter, followed by any number of letters or digits.
4. YACC program to evaluate an arithmetic expression involving operators +, -, * and /.
5. YACC program to recognize strings *aaab*, *abbb*, *ab* and *a* using the grammar ($anbn, n \geq 0$).
6. Program to recognize the grammar ($anb, n \geq 10$).

7.7 Day 7: Exercise Using C

1. Implementation of Predictive Parser.

8 Syntax Directed Definition

8.1 Day 8: Exercise Using C

1. Write a C program to implement the syntax-directed definition of **if E then S1** and **if E then S1 else S2**.
2. Write a yacc program that accepts a regular expression as input and produce its parse tree as output.

9 Generation of Intermediate Code

9.1 Day 9: Exercise Using C

1. Write a program for generating for various intermediate code forms: A Program to Generate Machine Code.
 - (a) Three address code
 - (b) Quadruple

2. Write a program to generate the intermediate code in the form of Polish Notation

10 Advanced Problems

1. Develop a recursive decent parser
2. Write a program to Simulate Heap storage allocation strategy
3. Generate Lexical analyzer using LEX.
4. Generate YACC specification for a few syntactic categories.
 - (a) rogram to recognize a valid arithmetic expression that uses operator +, - , * and /.
 - (b) Program to recognise a valid variable which starts with a letter followed by any number of letters or digits.
 - (c) Program to recognise the gramar(anb where $n \geq 10$)
 - (d) Implementation of Calculator using LEX and YACC

11 Books to Follow

- 1 John R. Levine, Tony Mason, Doug Brown; **Lex & Yacc**, *O'Reilly & Associates* 1992. ISBN: 9781565920002,
- 2 Charles N. Fischer, Richard J. LeBlanc Jr., Ron K. Cytron, **Crafting A Compiler** 2011, *Pearson Education*. ISBN: 9780133001570,

Evaluation Scheme

EC No.	Evaluation Com- ponent	Duration	Weightage	Data & Time	Nature of Component
--------	---------------------------	----------	-----------	----------------	------------------------

You May Meet Me: Every day 5:00pm.

You may mail me at bhaskarmondal.cs@gmail.com; (always mention your Roll Number followed by Subject at the subject field.)